

# Explanation for the Sentivity Analysis Module Code

Yunru Zheng

January 12, 2026

## 1 Revisiting previous mission

To briefly tie up loose ends from last time, we attempted parameter  $a, b, c, d = 1.0, 0.45, 2.0, 0.15$ , and I was able to plot the distribution on top of the histogram and they match.

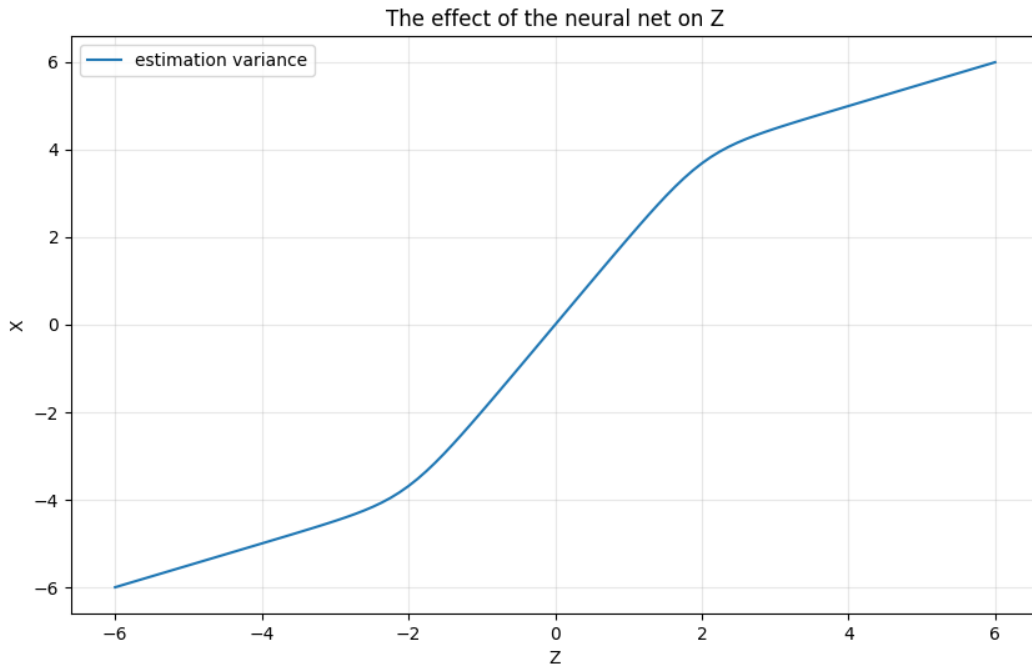


Figure 1: The effect of the neural net on  $Z$ .

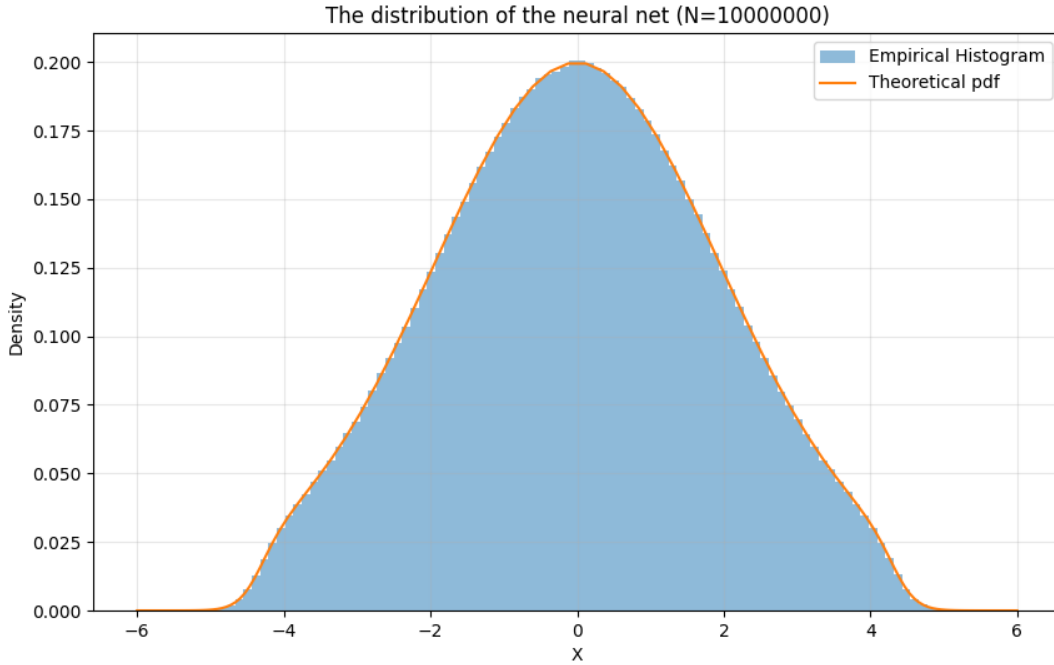


Figure 2: The distribution of the neural net for  $a = 1.0, b = 0.45, c = 2.0, d = 0.15$ .

## 2 Implementation

The implementation of Robbins Monro itself is ok and only takes some time (I may need more time to modularize it and add more notes). The really tricky part is setting up the constraints to make sure  $X$  doesn't just cluster around the center to make the variance small (I cannot replicate the mistakes I made but it was basically a very large  $a$  with small  $c$ ). The first few constraints that I set up was:

- $b, d > 0$  since the width is certainly positive, and we want the slope after the turn to be smaller than the initial slope
- $2a - b/d > 0$  to make sure that the neural net is one-to-one.

This doesn't give me very good results since it just outputs very large  $a$  (around 70!) and very small  $c$  so all the mass clusters around 0. I proposed new constraints:

- $c > 2.0$  (2.5 was also attempted) since for the standard gaussian 2 is 2 standard deviation away from the mean
- $a > 1.0$  since the initial slope should be big to move the weight towards the tail
- $d > 0.1$  (if  $d$  is too small it can push  $a$  to be super large)
- $2a - b/d > 1$  (debatable)

For the last constraint, I tried it as a constraint for  $b$  and  $a$ . Restricting  $b < d(2a - 0.5)$  makes the parameter stagnant. Restricting  $a = 0.5 + b/2d$  gives better results (not really better will explain in the next section).

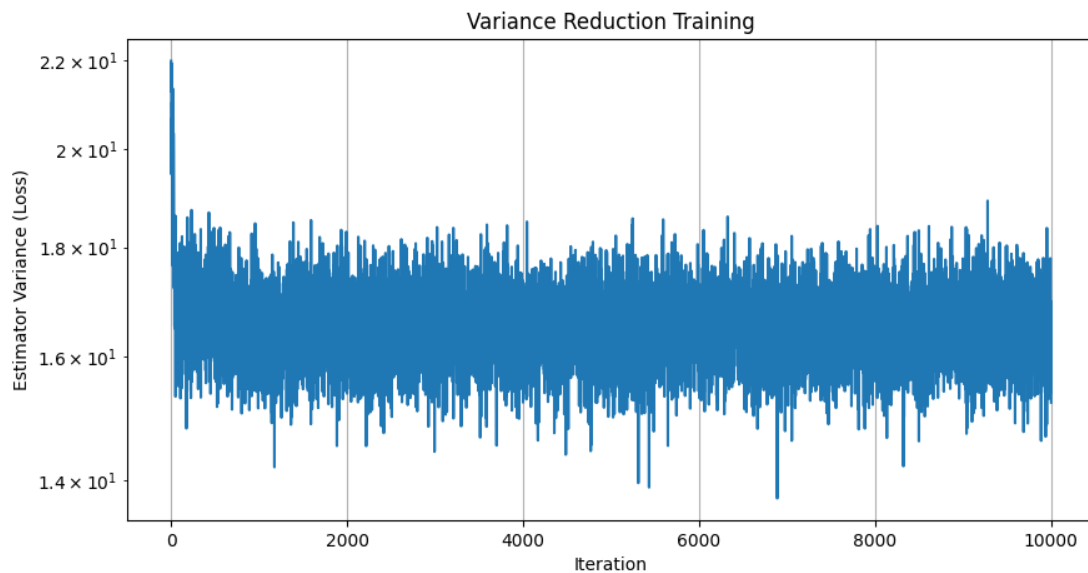


Figure 3: Loss after iterations restricting  $b$ .

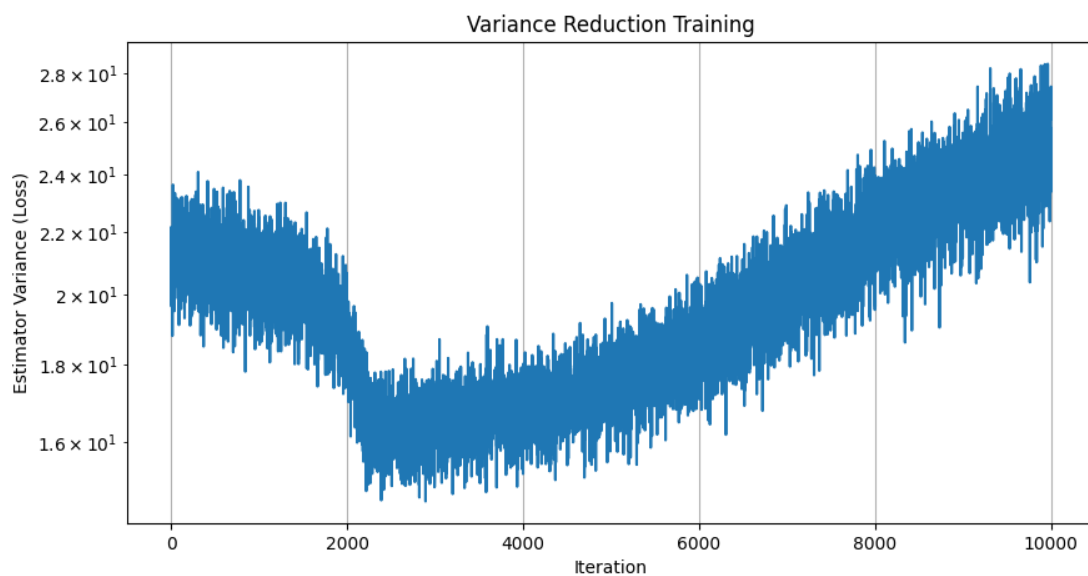


Figure 4: Loss after iterations restricting  $a$ .

### 3 Results

I started the optimization from  $a, b, c, d = 2.0, 0.45, 2.0, 0.15$ . I used batch size 1000 and did 1000 iterations with learning rate 0.005. I stuck to  $Z^4$  as the function, and assumed that  $Z$  is standard Gaussian. The neural net is defined similar to last time.

Unfortunately, I wasn't able to generate a batman graph, but instead generated an importance function similar to our previous sensitivity analysis, a gaussian with larger standard deviation. The failure is also partially seen from Figure 4 as you can see a decrease in loss and then gradual increase. The optimizer was working initially reducing the loss. However, in general the optimizer pushes for bigger  $a, b, d$  and values shoot up high and led to this result. (I also tried early stop. Unfortunately when loss is smallest the graph is still not ideal).

```
Iter 0: Loss=21.0075 | Theta=[1.9987931  0.45000526  2.5      0.15012254]
Iter 0: Loss=21.0075 | Estimate=2.9549
Iter 1000: Loss=20.5236 | Theta=[1.9064329  0.4503249  2.5      0.1600947]
Iter 1000: Loss=20.5236 | Estimate=2.9454
Iter 2000: Loss=19.0314 | Theta=[1.5908113  0.45263562  2.5      0.2074766 ]
Iter 2000: Loss=19.0314 | Estimate=3.0555
Iter 3000: Loss=16.1399 | Theta=[1.3253045  1.4828215  2.5      0.89834815]
Iter 3000: Loss=16.1399 | Estimate=2.9738
Iter 4000: Loss=18.0339 | Theta=[1.617623  2.9301548  2.5      1.310887 ]
Iter 4000: Loss=18.0339 | Estimate=3.1428
Iter 5000: Loss=17.8033 | Theta=[1.9372643  4.5251274  2.5      1.5742154]
Iter 5000: Loss=17.8033 | Estimate=3.0645
Iter 6000: Loss=19.6559 | Theta=[2.3322878  6.3265595  2.5      1.72641 ]
Iter 6000: Loss=19.6559 | Estimate=3.1342
Iter 7000: Loss=20.7434 | Theta=[2.7643144  8.183564  2.5      1.8070734]
Iter 7000: Loss=20.7434 | Estimate=3.0713
Iter 8000: Loss=22.1189 | Theta=[3.1917048  9.953607  2.5      1.848941 ]
Iter 8000: Loss=22.1189 | Estimate=3.0657
Iter 9000: Loss=23.9012 | Theta=[ 3.5789652 11.524436  2.5      1.8714788]
Iter 9000: Loss=23.9012 | Estimate=3.0801
Final Theta: [ 3.922966 12.910944  2.5      1.8859293]
```

Figure 5: Run results after 10000 iterations with batch size 1000.

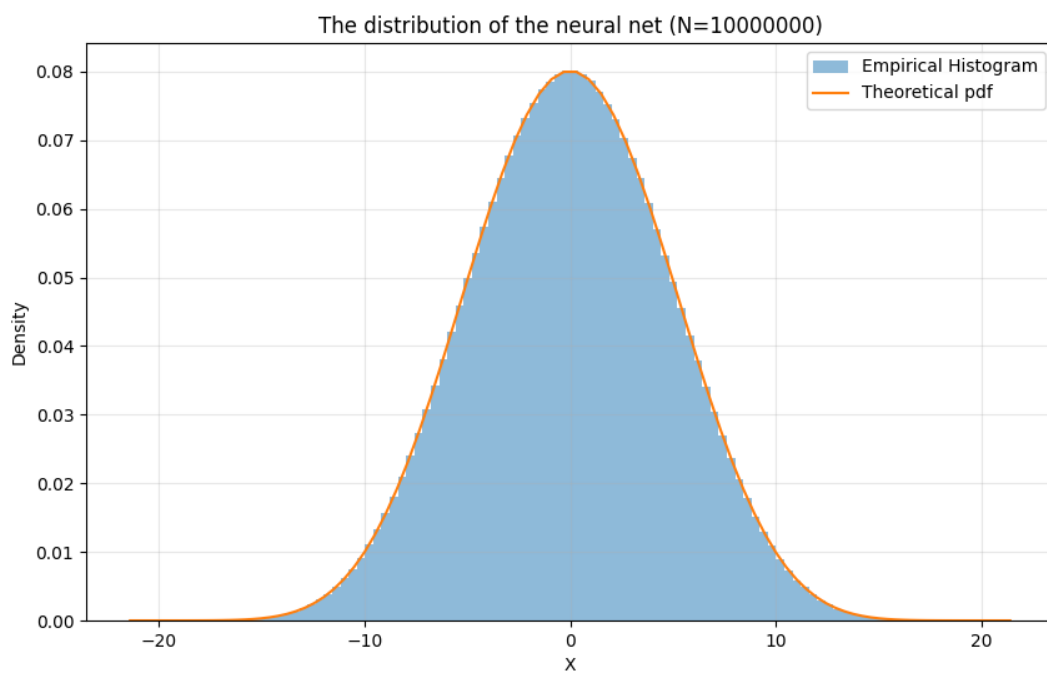


Figure 6: The distribution of  $a = 3.92, b = 12.91, c = 2.5, d = 1.89$ .